CG2271 Real Time Operating Systems

2026/27 Semester 1

Interim Report

Seat Guardian: Smart Library Seat Monitor

B01_YY	
Name	Matric Number
Yan Xiangyu	A0305605M
Goh Shao Ann	A0307779L
Tay Guang Sheng	A0273178W
Kothari Smeet Ronak	A0311491M
Tejas Ananth Kumar	A0281316J

Problem statement

Students often "chope" library seats with their belongings and leave for hours, while noisy neighbours go unchecked, so scarce seats are wasted and quiet zones are disturbed. Seat Guardian monitors a library seat, detects when it has been abandoned or is too noisy, and lets an LLM-based assistant decide what to do, so a user can legitimately hold a seat ("back in 30 min") while librarians receive data-driven daily reports.

Sensors and Actuators used

- **Ultrasonic sensor HC-SR04 (MCXC444, polled):** measured every 500 ms by a periodic task to detect whether someone is sitting at the seat.
- **Sound sensor, digital output (MCXC444, interrupt):** a rising edge raises an interrupt for each loud noise event; the ISR only counts and signals a task.
- **Touch sensor KY-036 (MCXC444, interrupt):** the user taps it to check in or confirm "I'm back", which resets the away timer.
- **Photoresistor LDR (ESP32, polled via ADC):** records lighting quality at the seat for logging and for the LLM's reports.
- **Onboard RGB LED (MCXC444, PWM):** shows the seat state: green free, blue occupied, amber away/on hold, red abandoned, flashing on noise.
- **Buzzer (MCXC444, PWM):** short chirps warning the user before a seat is released and when noise exceeds the limit.
- **Servo SG90 (ESP32, PWM):** flips a physical "RESERVED / FREE" flag on the desk when a seat is held or released.

General Architecture

Hardware. The MCXC444 handles all time-critical sensing and local feedback; the ESP32 handles Wi-Fi, the LDR and the servo. The boards are linked by UART (MCXC444 UART2 on PTE22/PTE23, ESP32 UART1) with a common ground; the 5 V HC-SR04 echo line goes through a voltage divider.

MCXC444 software (FreeRTOS, pre-emption and time slicing enabled). NoiseISR and TouchISR only give a binary semaphore; the UART2 ISR assembles incoming bytes into packets and posts them to a command queue with xQueueSendFromISR. SensorTask polls the ultrasonic sensor and sends readings to a sensor queue. SeatTask (highest priority) runs the seat state machine (FREE → OCCUPIED → AWAY → ON HOLD / ABANDONED) from the queue and semaphore events. ActuatorTask drives the LED and buzzer from ESP32 commands, and UartTxTask (lowest priority) packetises status updates. A mutex protects the shared seat-state structure and another protects the UART transmit buffer.

Communication protocol. Instead of plain text lines, both boards exchange framed packets [0xAA | type | length | payload | XOR checksum]. The receiver replies ACK or NAK and the sender retransmits on NAK or timeout, so corrupted frames are detected and the link resynchronises.

Cloud and AI services (ESP32). The ESP32 runs separate tasks for UART, LLM requests, Telegram and Firebase so that slow HTTPS calls never block the board-to-board link. It uses three online functions:

- **LLM seat decision:** on each state change the ESP32 sends {occupied, minutes_away, noise_events, lux, time} to the NUS SoCLaaS chat-completions API, with the library's seat rules in the system prompt. The model returns JSON {action: FREE | WARN | RESERVE, reason}, which is validated and forwarded to the MCXC444. If the API times out, a local rule-based fallback keeps the seat working.
- **Natural-language Telegram assistant:** users message a Telegram bot in plain English, e.g. "going for lunch, hold my seat for 30 mins". The LLM converts the message into a structured command ({intent: hold, minutes: 30}, status, release or complaint), the ESP32 sends it to the MCXC444 (LED turns amber, servo shows RESERVED) and the bot replies with the result.
- **Firebase logging and daily LLM report:** every event and decision is written to Firebase Realtime Database. Once a day the ESP32 reads back the day's log and asks the LLM to summarise it for librarians (peak chope hours, noisiest periods, suggested rule changes); the report is delivered through Telegram.

Current progress

From Lab 3 we already have the foundation of the communication layer: an interrupt-driven UART2 driver on the MCXC444 that passes received messages to FreeRTOS tasks through a queue, and an ESP32 sketch that reads an analog light sensor and exchanges messages with the MCXC444 over UART. Both toolchains are set up, and the components, task structure and packet format above are decided.

Plan: Week 10, sensor drivers and seat state machine; Week 11, packet protocol and SoCLaaS seat decision; Week 12, Telegram assistant, Firebase logging, daily report and servo; Week 13, integration, demo and final report.